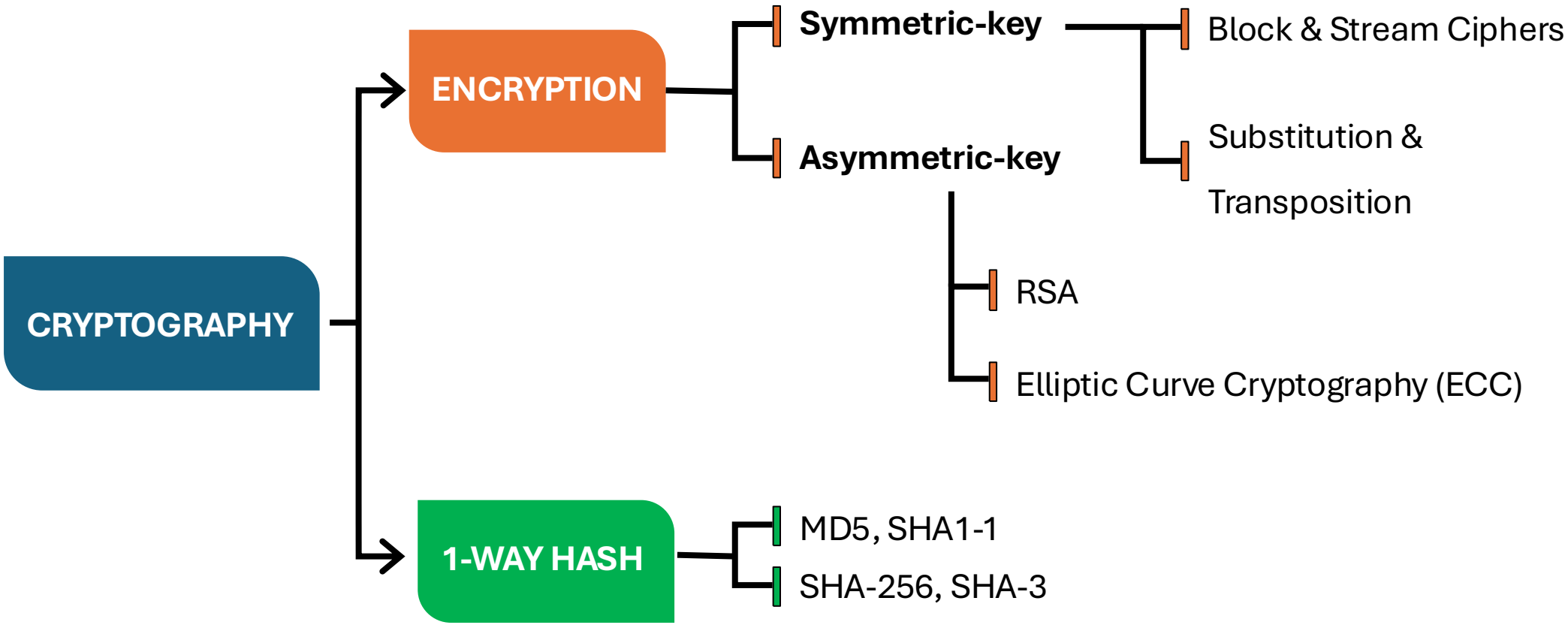
Lecture 5:
Hash Function, MAC, Digital Signature

ITCS312: Computer and Communication Security

Dolvara Gunatilaka, PhD

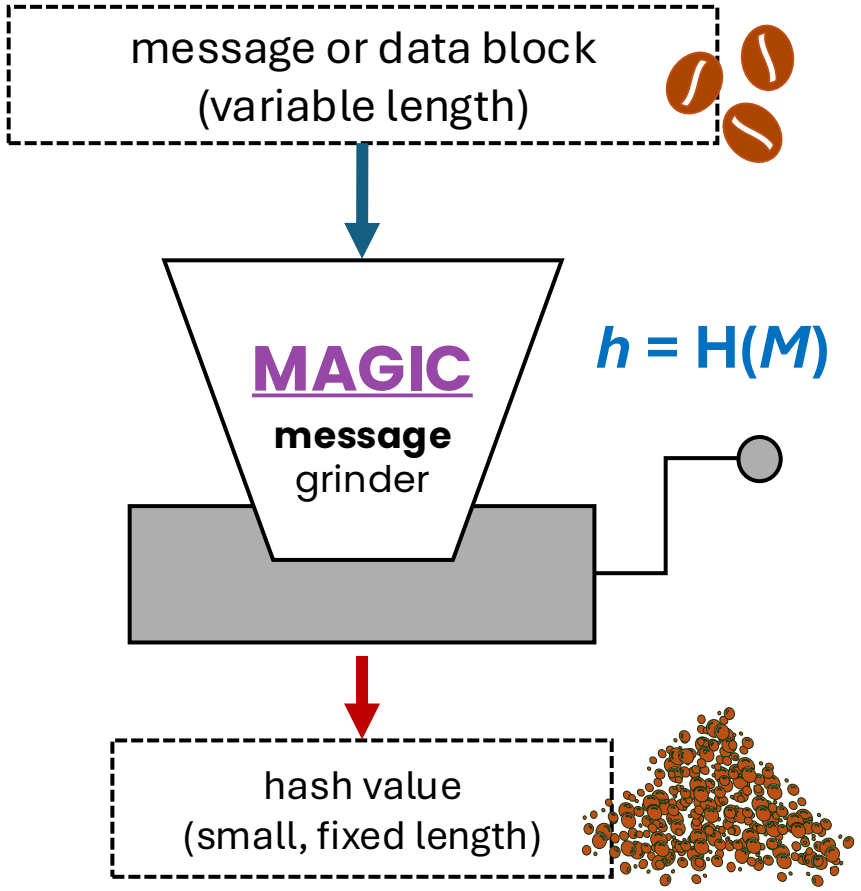
Classification of Cryptographic Systems



Topics

- **Intro to Cryptographic Hash Function**
  - **Hash Function Applications**
  - **Cryptographic Hash functions: MD and SHA**
- Message Authentication (MA)
  - Message Authentication: MAC, HMAC
  - Attacks on hash functions: Pigeon Hole Principle and Birthday Attack
- Digital Signature and Digital Certificate

One-Way Hash Function



One-way Coffee Grinder!

- **One-way:** it is infeasible to reverse the process (irreversable)
- **Deterministic:** the same input will always produce the same output
- **Hash function:** condenses **variable length input** to **fixed length output** called hash code or digest
- Usually public (open design!!!)
- Used to detect changes in message

One-Way Hash Function

What makes a <u>good</u> cryptographic hash function?

- Computationally <u>infeasible</u> to find data mapping to a specific hash
  - ***One-way*** *property*
- Computationally <u>infeasible</u> to find two data mapping to the same hash
  - ***Collision-resistance*** *property*
- A small change in input (even one bit) should drastically change the hash output.
  - ***Avalanche Effect***

Application of Cryptographic Hash Functions (Examples):

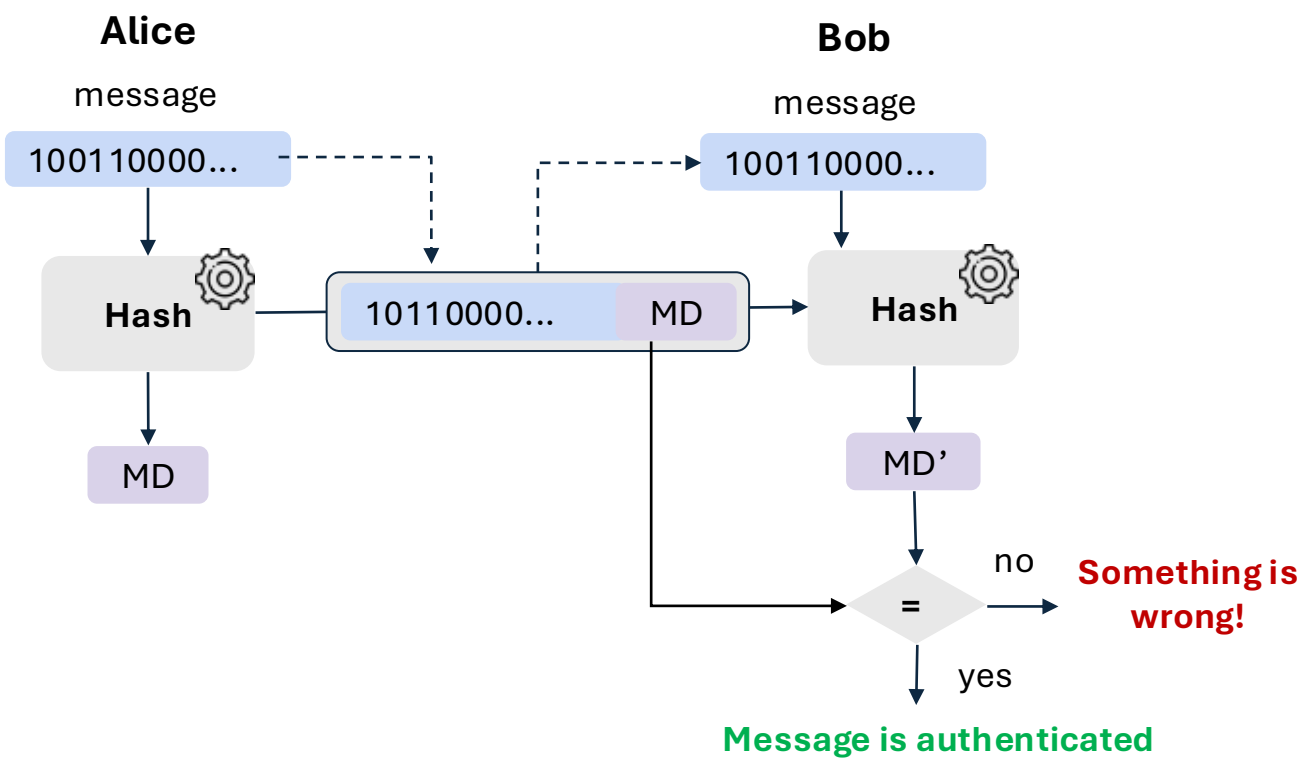
- Message Integrity Check
- Message Authentication (MA) (Data Integrity Verification)
- Digital Signatures
- & some other uses

One-Way Hash: Other Uses

- **Storing Password**
  - Store hash value of password not actual password in the database
- **Intrusion and Virus Detection**
  - Keep and verify hash values of files on the system
  - Malware Signature
- **File Comparison**
  - If two files produce the **same hash**, they are likely the same (with high probability).
- **Pseudorandom Number Generators (PRNGs)**
  - Transform counter or timestamp into pseudorandom output
- **Blockchains**
  - Used to link blocks together

One-Way Hash: Message Integrity Check

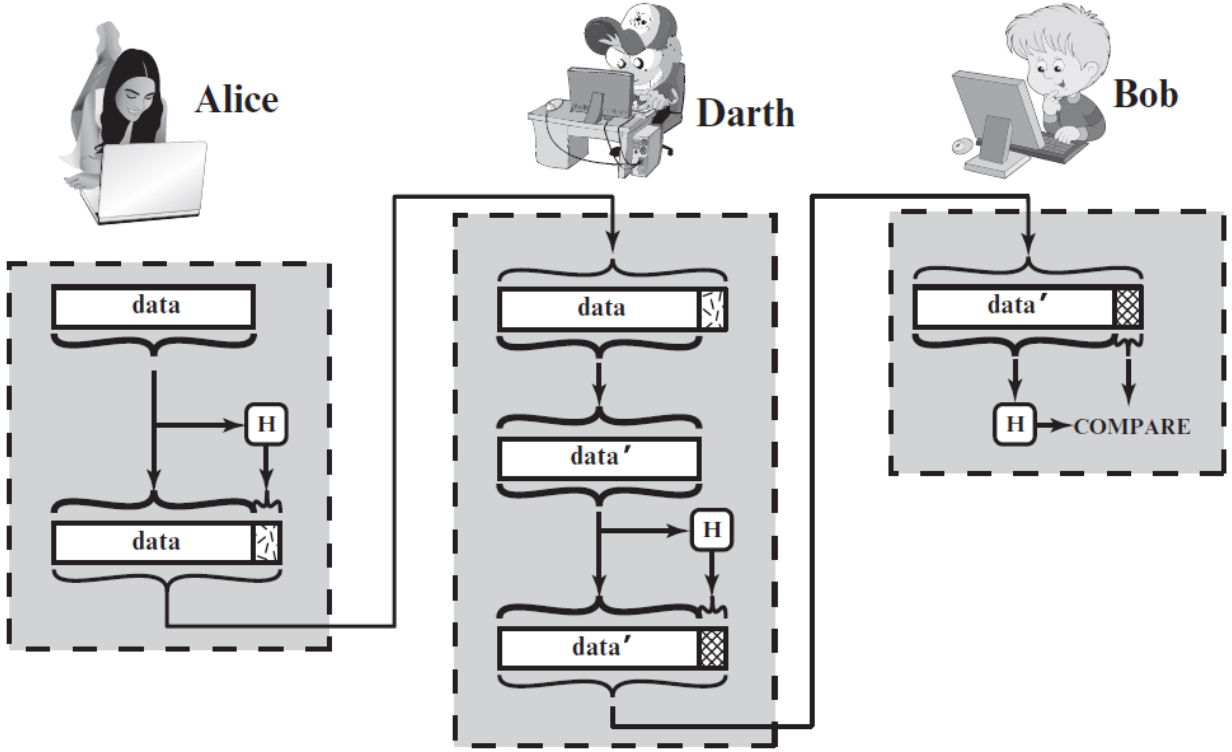
- Mechanism of service used to verify the integrity of message
- Assure that data received are exactly as sent
- Hash value (or hash code) is sometimes referred to as message digest (MD)
- Requirement: identity of sender is valid!



One-Way Hash: Message Integrity Check

Limitation:

- A simple hash value only verifies data integrity, **<u>not</u> the sender's identity**
- An attacker could replace both the message and its hash
  - ***Man-in-the-Middle attack (MITM)***

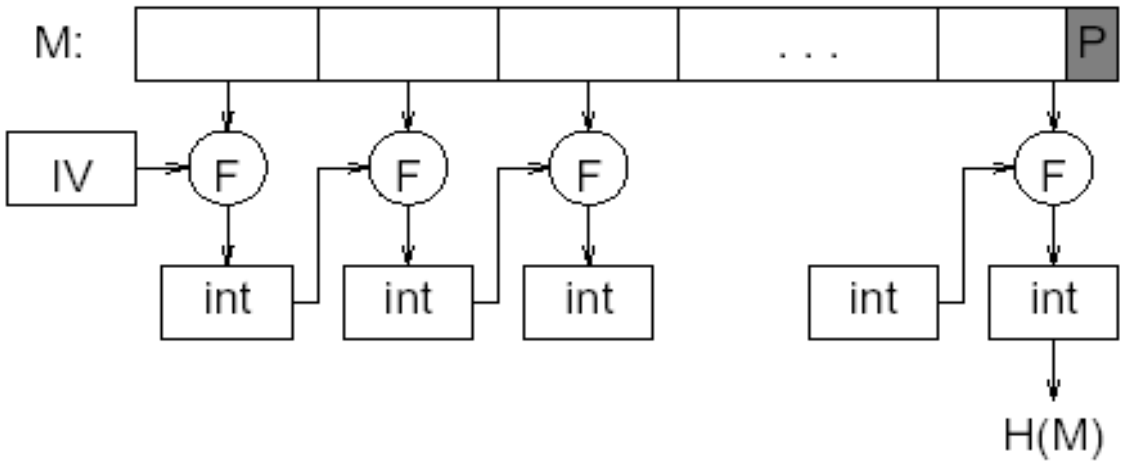


Message Digest (MD)

- MD is a hash function designed by *Ron Rivest*
- Only for data ***integrity*** check
- An output hash value, called ***message digest (MD),*** is generated using a 1-way hash function:

MD = H(M)

- *No* ***secret-key*** involves



Message Digest (MD): MD Family

- **MD2:** (1989) produces a 128-bit hash value, perceived as slower and less secure than MD4 and MD5
- **MD4:** (1990) produces a 128-bit hash of the message, using bit operations on 32-bit operands for fast implementation, influenced on SHA-1, RIPMEND
- **MD5:** (1992) produces a 128-bit output, was widely used.
  - Commonly used for checksum
  - Fast, but not collision resistant
- **MD6:** (2008) proposed to NIST as SHA-3 candidate, improved MD4-5, designed for parallel computation for very long input.
  - Variable digest size (up to 512 bits)
  - Stronger security, but slow

Secure Hash Algorithm (SHA): SHA-0, SHA-1

- **SHA (Secure Hash Algorithm) family** is a group of cryptographic hash functions developed and standardized by **NIST**
- **SHA** (SHA-0) originally designed by **NIST** & **NSA** in 1993 → obsolete
- Was revised in 1995 as **SHA-1**
  - Produce **160-bit hash values**
  - Practical **collision attacks** have been demonstrated
  - **No longer secure and deprecated**

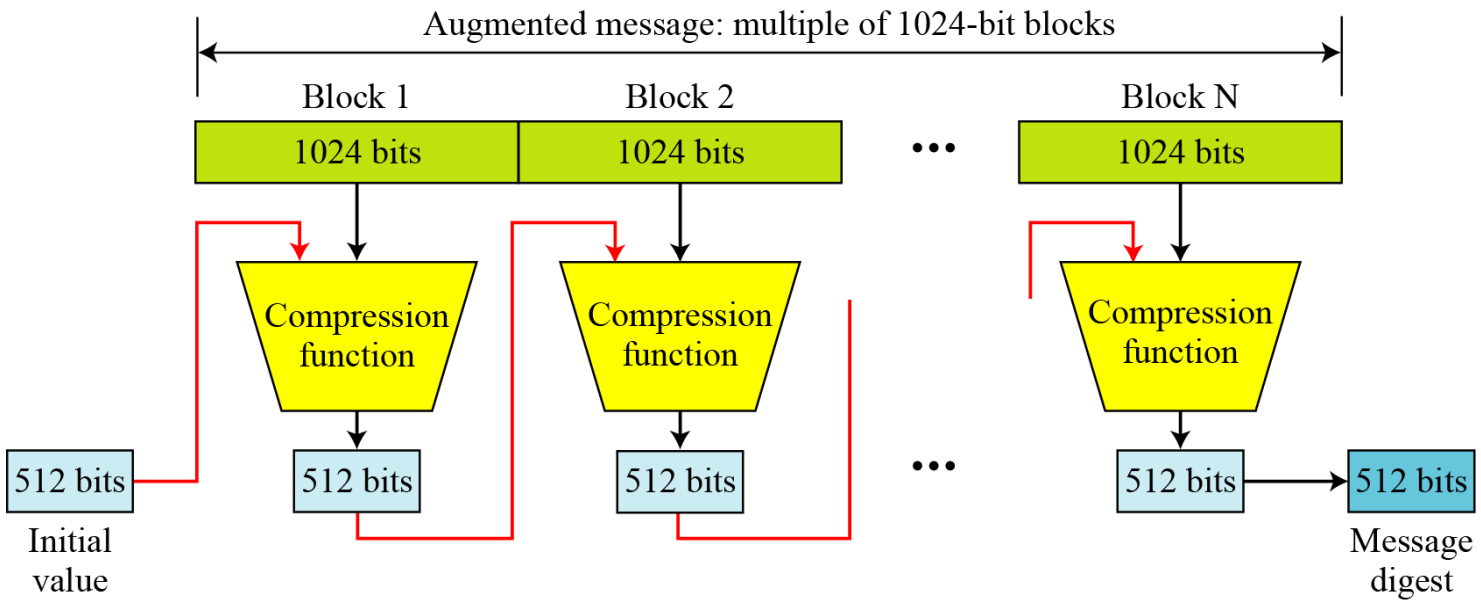
Secure Hash Algorithm (SHA): SHA-2

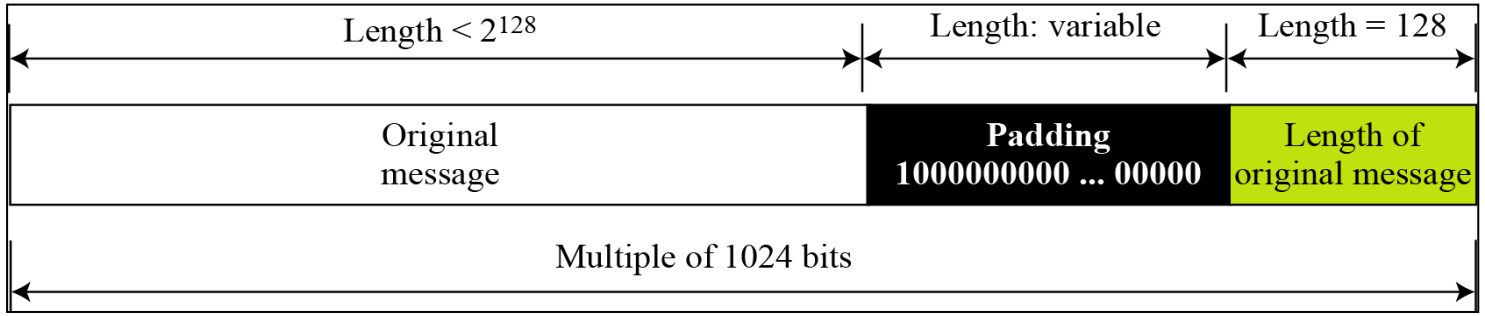
- **SHA-2** published in 2001 is a major revision
- Consits of a family of hash functions: **SHA-224, SHA-256*, SHA-384, SHA-512***
  - Key difference is the size of the message digest
- Provide *higher level of security* than SHA-1
- *SHA-2 remains secure and viable*

		SHA-2 Family			
	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Output size	160	224	256	384	512
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block size	512	512	512	1024	1024
Word size	32	32	32	64	64
No. of steps	80	64	64	80	80

Secure Hash Algorithm (SHA): SHA-512

Merkle–Damgård construction uses in **SHA-1** and **SHA-2** family





Secure Hash Algorithm (SHA): SHA-3

- **SHA-3** cryptographic hash function **standardized in 2015** by NIST, based on the **Keccak** algorithm
- Propose as SHA-2 alternative, sometime can be used as a direct replacement of SHA-2
- Uses a **sponge construction**, making it structurally and internally very different from SHA-1 and SHA-2
- Offers <u>different output sizes</u>, including 224, 256, 384, and 512 bits

Topics

- Intro to Cryptographic Hash Function
  - Hash Function Applications
  - Cryptographic Hash functions: MD and SHA
- **Message Authentication (MA)**
  - **Message Authentication: MAC, HMAC**
  - **Attacks on hash functions: Pigeon Hole Principle and Birthday Attack**
- Digital Signature and Digital Certificate

Message Authentication

- **Message authentication** is a cryptographic mechanism ensuring that
  - A digital message's origin is verified **(authentication)**
  - It has not been tampered with in transit **(integrity)**
- **Message Authentication Code (MAC):**
  - A small, fixed-size block of data (called “tag”)
  - Computed using a ***symmetric key***
  - Appended to the message for verification.
- Only parties who know the key can generate or verify a valid MAC.

MAC = C(*K*,*M*)

where:

M = input message

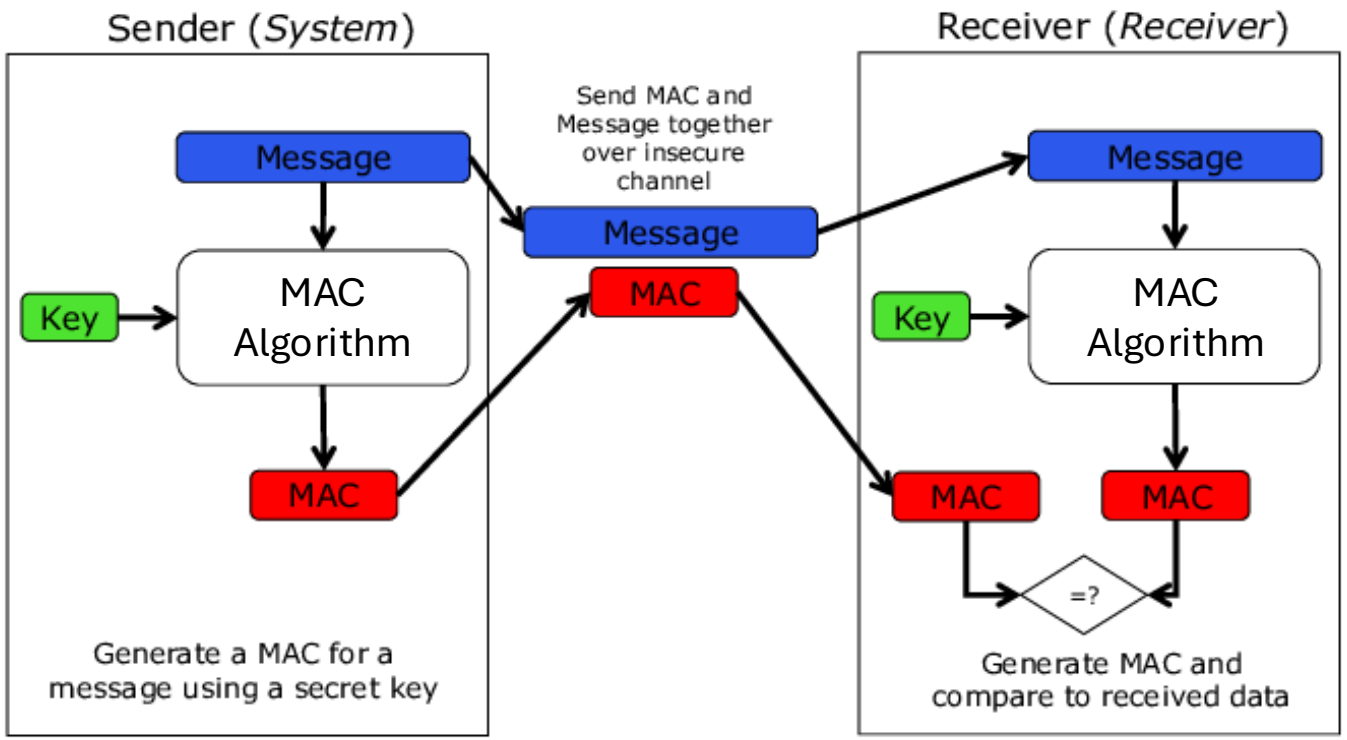
C = MAC function

K = shared secret key

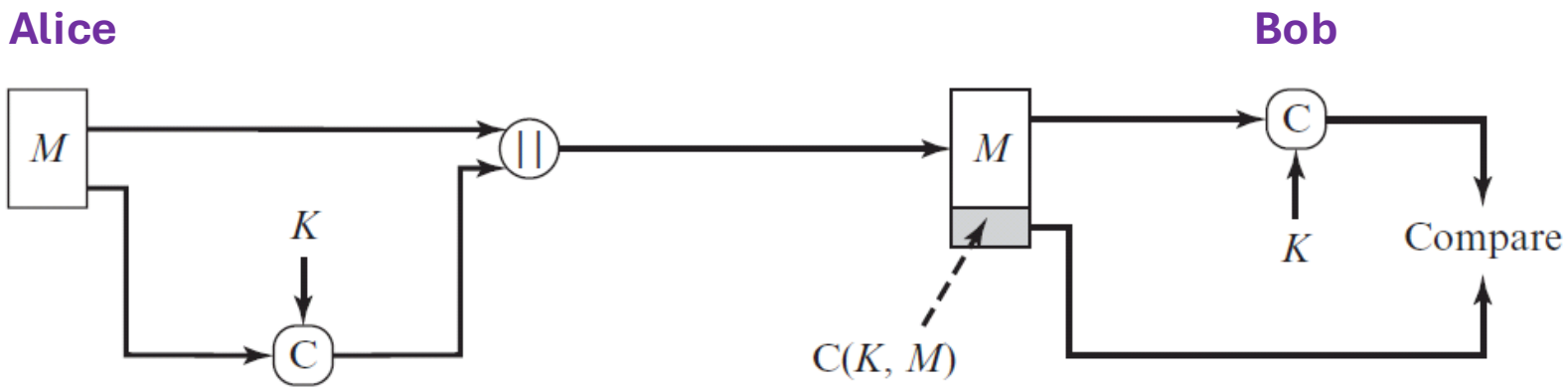
MAC (output)

Message Authentication Code

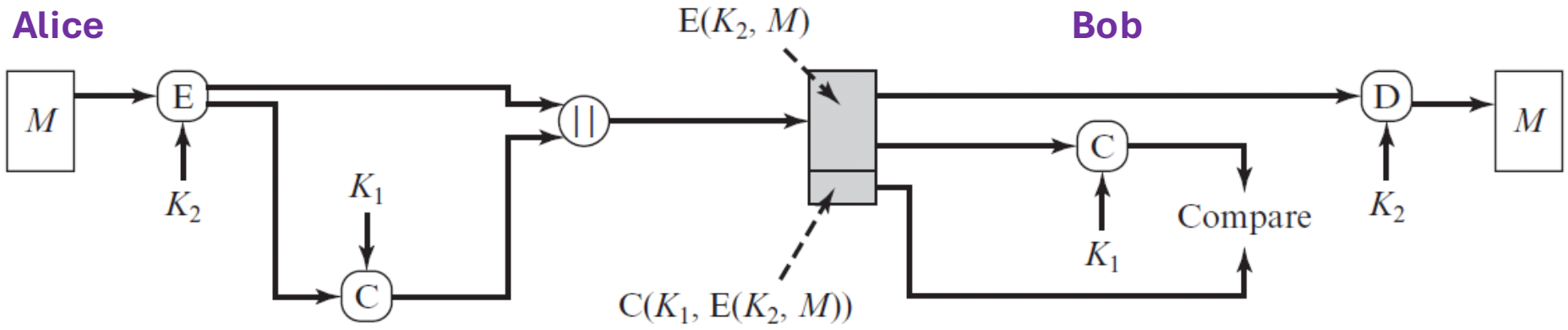
- MAC provides <u>assurance that</u>
  - **Message is unaltered**
  - **Comes from the sender who has the key**
- Common MAC algorithms
  - **CBC-MAC**: based on block ciphers, legacy
  - **CMAC**: AES-based, standardized
  - **HMAC**: hash-based, most widely used



Message Authentication Code (MAC)

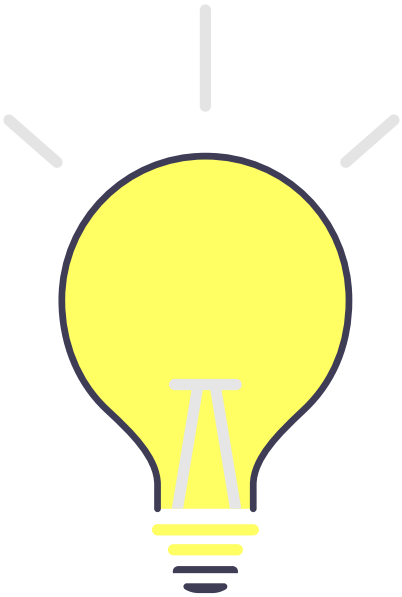


MAC is computed from plaintext



MAC is computed from encrypted message

Hash-Based MAC (HMAC)



- HMAC is a <u>specific</u> construction for a <u>MAC</u> involving a cryptographic ***hash function*** and a ***secret cryptographic key***
  - Hash functions are generally faster
  - Cryptographic hash function code is widely available
- Combines the key and message using **inner and outer hashing**
- Commonly uses **SHA-256 or SHA-512**
- Stronger than basic MACs
- Extremely widely deployed (TLS, IPSec, and other protocols)

How HMAC Works?

- Specified as Internet standard RFC2104

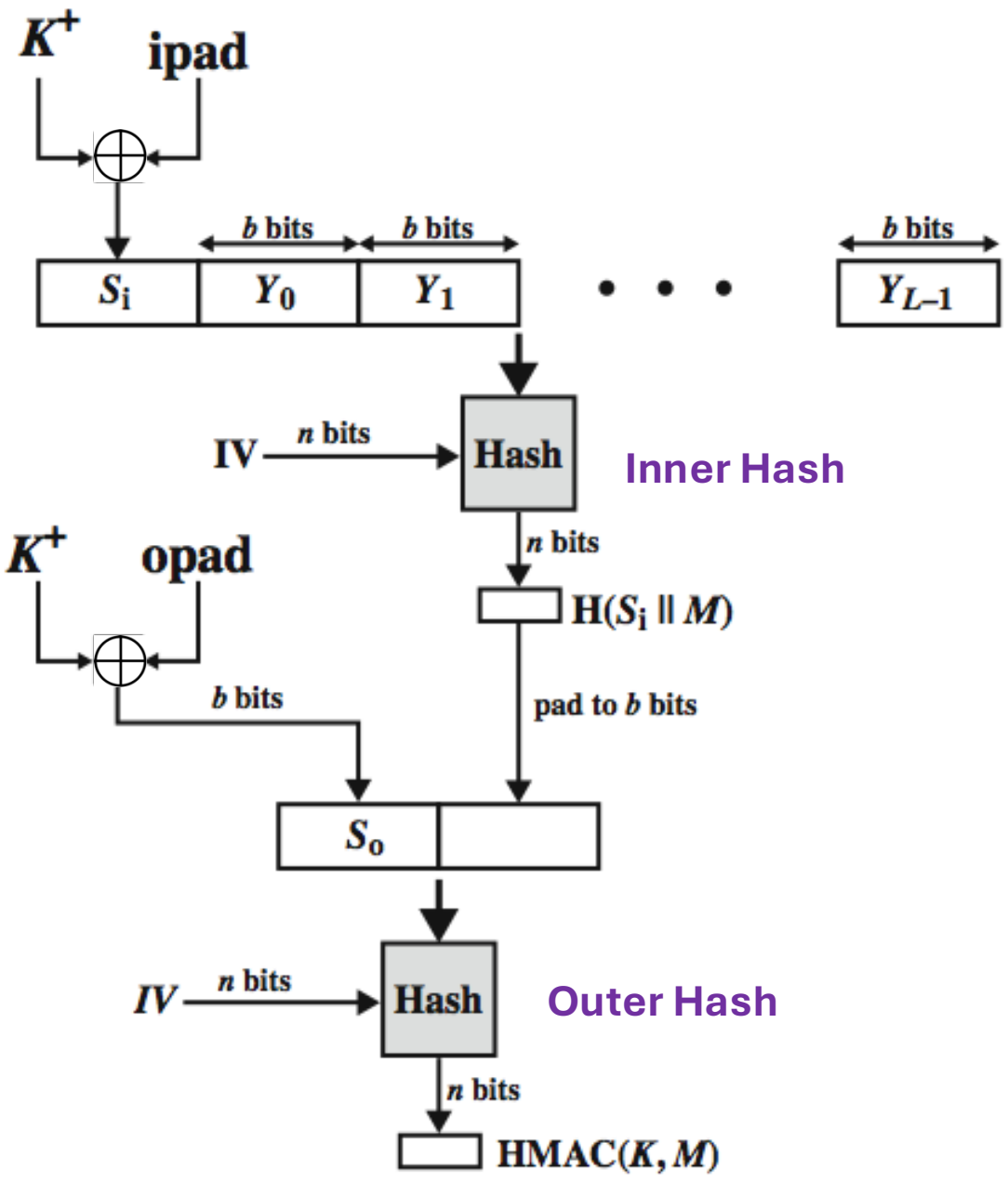
$$\mathbf{HMAC(K, M) = H[\ (\mathit{K}^{+} \oplus opad) \ || \ H[\ (\mathit{K}^{+} \oplus ipad) \ || \ \mathit{M}\ ]\ ]}$$

  - ***K*** = shared secret key
  - ***M*** = message
  - K^{+} = Secret key padded with 0's to fill the block
  - ***opad***, ***ipad*** are specified padding constants

- Overhead (cost) is 2 hash calculations
- Any hash can be used (e.g., MD5, SHA-2, RIPEMD-160, Whirlpool, etc.)

How HMAC Works?

- ipad = 00110110 (repeated to fill the *b*-bit block)
- opad = 01011100 (repeated to fill the *b*-bit block)
- K^+ = key padded with 0s resulting in b-bit length



HMAC Security

- Security of HMAC relies on its underlying hash algorithm
- HMACs do not provide **non-repudiable**; the sender and receiver share the same key, so you can't prove who created the message
- Secret keys must be securely exchanged/stored and mismanagement undermines security
- Inner and outer hash with different padding makes known-hash attacks difficult.
- Attacking HMAC requires either:
  - **Exhaustive key search** (brute-force attack)
  - **Birthday attack** (not easy, since HMAC requires key)

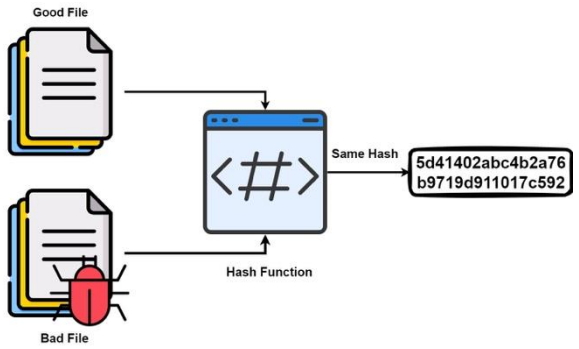
Attacks on Hash Function

Brute-force attack (always theoretically possible!)

- **Preimage attack**
  - Given a target hash value of m bits, find the original plaintext
- **Second preimage attacks**
  - Given a message **x** find a message y such that H(y) = H(x)
- Usually takes 2^{n-1} tries (half distance searching) before succeeded
  - n = no. of bits in hash output

Collision attacks

- Find **ANY** 2 messages ***x*** and ***y*** with the same hash H(*x*) = H(*y*)
- No specific message is given in advance
- Much *easier* than brute-force attack
- 128-bit → inadequate, 160-bit → being suspected



Pigionhole Principle

Given ***n holes for n+1 pigeons***, then at least 1 hole <u>will contain at least two pigeons</u>

Example:

- Let's assume the length of a message is 5 bits and output of hash function is 3-bit long.
- 2^5=**32 possible messages** will be mapped to 2^3= **8 possible hash values**
- Many messages will be <u>hashed into the same value</u> (collisions !)
- Real messages and hash values are much longer than 5 or 3 bits! But, **now we know that collisions exist**
- Only, it is **much tougher to find collisions** and **much tougher to forge** them

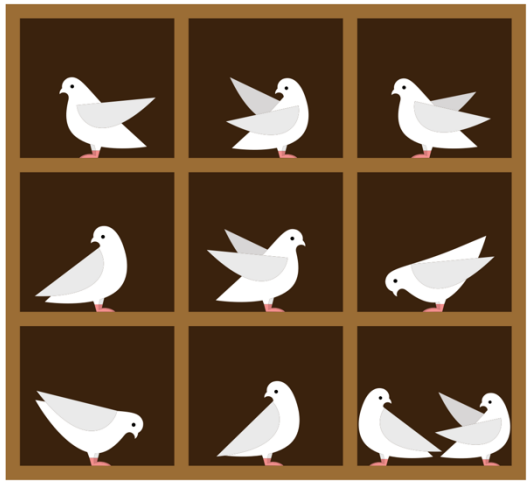


Image: https://gonitsora.com/certainty-problems-pigeonhole-principle/

Birthday Pradox

"Among 23 people, there is roughly ?? % chance that 2 people share the same birthday!"

- Chance that 2 people not sharing the same birthday = 364/365
- Chance that the first 2 people also not sharing the same birthday with the next guy = (364/365)*(363/365)
- with the next guy (22 times in total) = (364/365)*(363/365)*(362/365)*...
- In summary = $364!/(342!*365^{22})$ = 0.492703 → 0.507297

not match **match**

Birthday Attack

- Attackers try to generate many random meesages and hash them
- Find any two messages that produce the same hash → ***hash collision***
- The expected number of attempts to find a collision is $\sim 2^{n/2}$, n = number of bits of the hash output
- To prevent birthday attack, larger *n* is needed.
- In short, we need to use larger MAC/hash

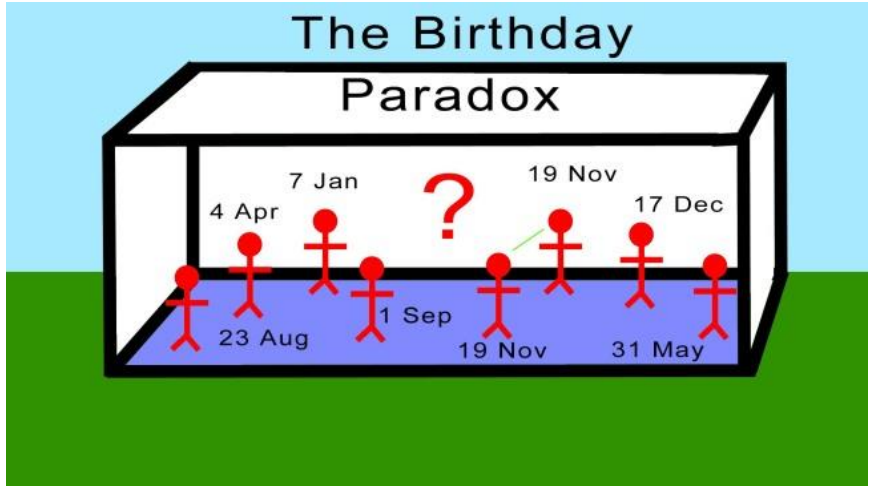


Image https://intuitivescienceblog.wordpress.com/2017/05/12/birthday-paradox-why-the-odds-of-same-birthdays-are-higher-than-we-think/

Topics

- Intro to Cryptographic Hash Function
  - Hash Function Applications
  - Cryptographic Hash functions: MD and SHA
- Message Authentication (MA)
  - Message Authentication: MAC, HMAC
  - Attacks on hash functions: Pigeon Hole Principle and Birthday Attack
- **Digital Signature and Digital Certificate**

Digital Signature

- Provide **integrity**, **authentication,** and **non-repudiation**
- Use a combination of cryptogrphic **hash function** and **asymmetric encryption**
- Digital signature is signed over a <u>hash of a message</u>
  - Hash the message
  - Sign the hash
  - Send the message

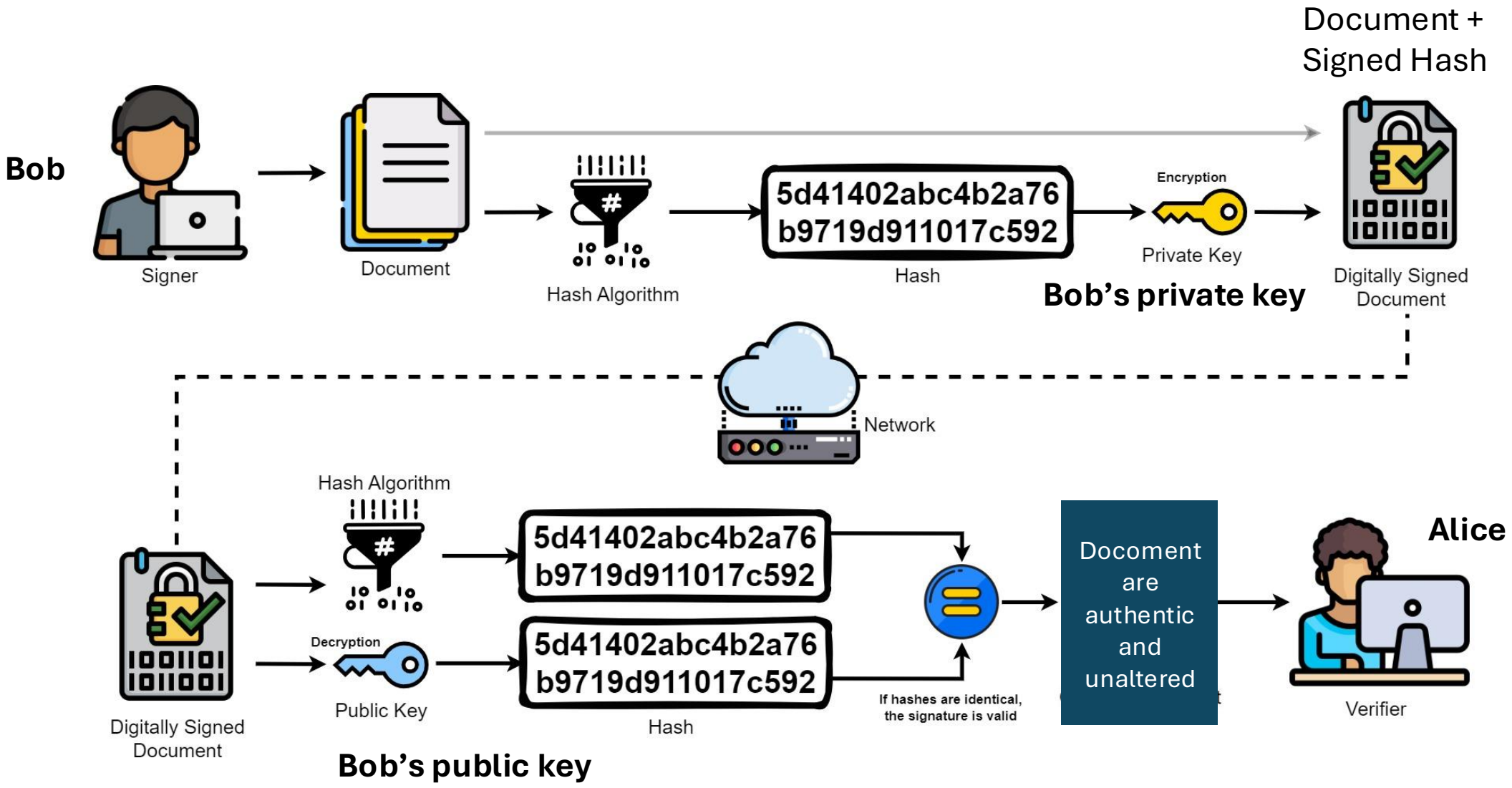
✅ **PKE for digital signature**

- The **sender signs** data using a **private key**
- Anyone can **verify** the signature using the sender's **public key**
- If verification succeeds, the message is trusted

✅ **Why Hashing Is Used**

- Signing the entire message is inefficient
- Hashing makes the process fast and secure
- Any change in the message changes the hash

How Digital Signature Work



What Makes a Good Digital Signature

1) Must *depend on the message* signed
2) Must use *information unique to sender*
   - <u>prevent</u> both **forgery** and **denial**
3) Must be relatively *easy to produce*
4) Must be relatively *easy to recognize & verify*
5) Be computationally *infeasible to forge*
   - Cannot construct a new message for a existing digital signature
   - Cannot construct a fraudulent DS for a given message
6) Be practical when keeping a copy of a digital signature in storage

Digital Signature Standard (DSS)

- DSS specifies a *suite of algorithms* that can be used to generate digital signatures established by NIST in 1994.
- Four revisions to the initial specification have been released in 1996, 2000, 2009, 2013.
- **FIPS 186-5: Digital Signature Standard (DSS) :** the current global standard supporting the following algorithms:
- **DSA (Digital Signature Algorithm):** original algorithm
  - Based on discrete logarithm problem
  - Fast signature generation; less commonly used today

DSS specifies:

- Key generation methods
- Signature generation and verification
- Approved Security parameters (e.g., key lengths, hash functions)

Digital Signature Standard (DSS)

FIPS 186-5. Digital Signature Standard (DSS) supported algorithms:

- **RSA Digital Signature**
  - RSA public-key cryptography (integer factorization)
  - Most widely used; slow signing, but very fast signature verification.
- **ECDSA (Elliptic Curve Digital Signature Algorithm)**
  - Based on elliptic curve cryptography
  - Modern security; provides high strength with much smaller keys (ideal for mobile/IoT) → fast and efficient
- **EdDSA (Edwards-curve Digital Signature Algorithm)**
  - Modern digital signature scheme
  - Fastest in signing, secure, and resistant to implementation errors
  - Increasingly popular (SSH, modern cryptography)

Problems with PKE Cryptosystem

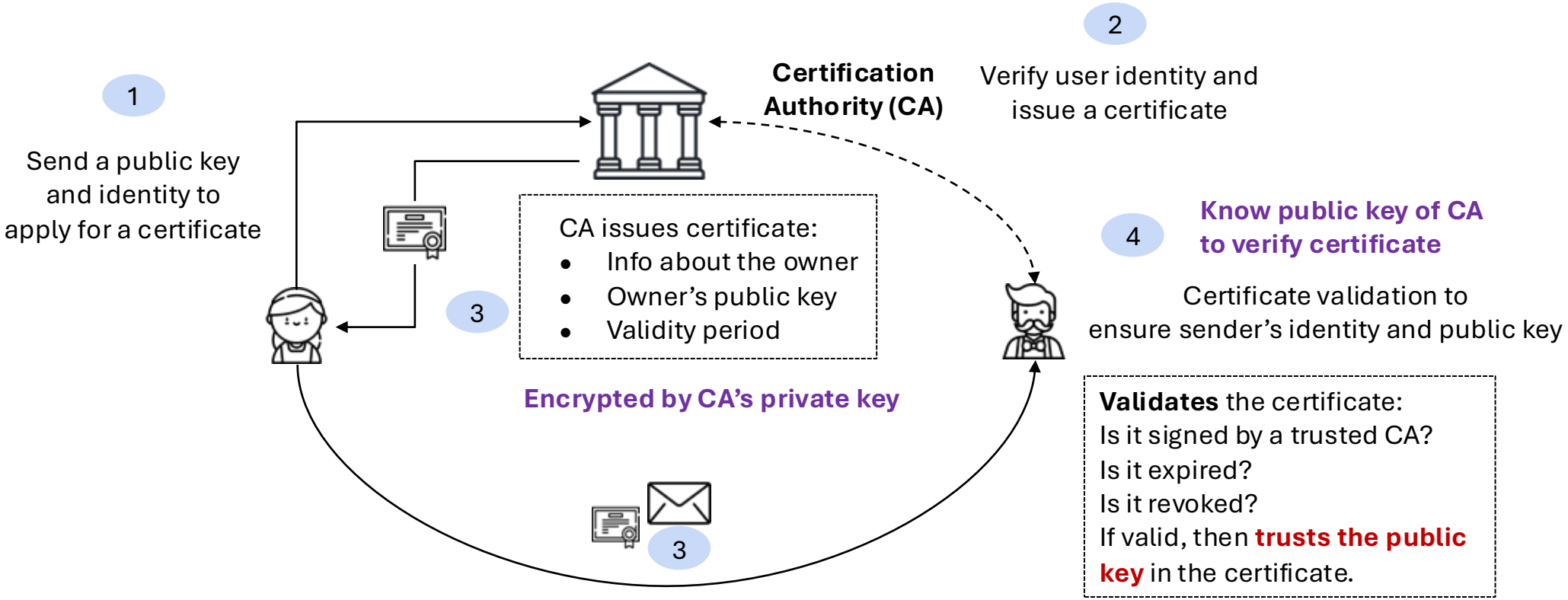
- Issue with Public Key Distribution:
  - ***How can we trust the public-key being told belong to the claimed person/server?***
- Main problem: man-in-the-middle attack that can lead to ***impersonation***
  - An attacker can give you *their* public key and claim: "This is Alice's public key"
  - If you trust it, the attacker can decrypt, read, or sign messages as Alice.
- Trust on the Internet?
  - Need Trusted Third Party (TTPs) → **Certificaet Authority (CA)**
  - These TTPs create ***certification entities*** in the cyberspace
  - E.g., **VeriSign, IdenTrust, Comodo, DigitCert, GoDaddy, GlobalSign**

Digital Certificate and Certificate Authorities

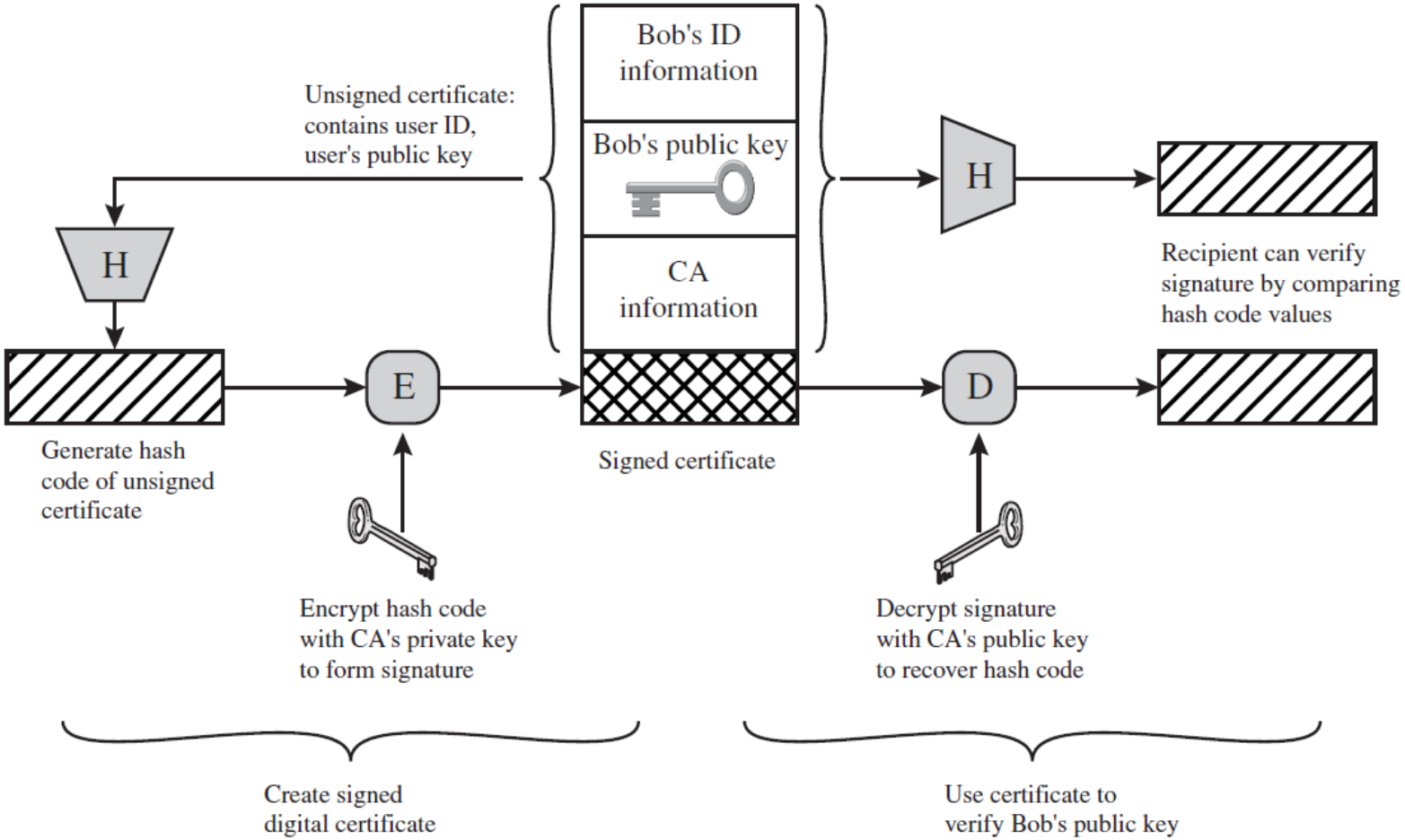
- **Certificate Authority (CA)** is a trusted party that generates **digital certificates**
- A **digital certificate** is an electronic document that:
  - Binds an identity to ***owner's public key***
  - Digitally signed by a trusted authority, i.e., ***CA's private key***
- CA certifies the ownership of a public-key by the named subject of the certificate
- All nodes need to be preconfigured with the ***CA's public key*** so that they can verify its signature on certificates
  - Certificate is issued and signed by private-key of CA
  - Be verified by anyone who knows the public-key of CA
- The format of these certificates is specified by the **X.509 standard**

Digital (Public-Key) Certificate

Digital certificate verifies the **identity** of a person, organization, or device on the internet



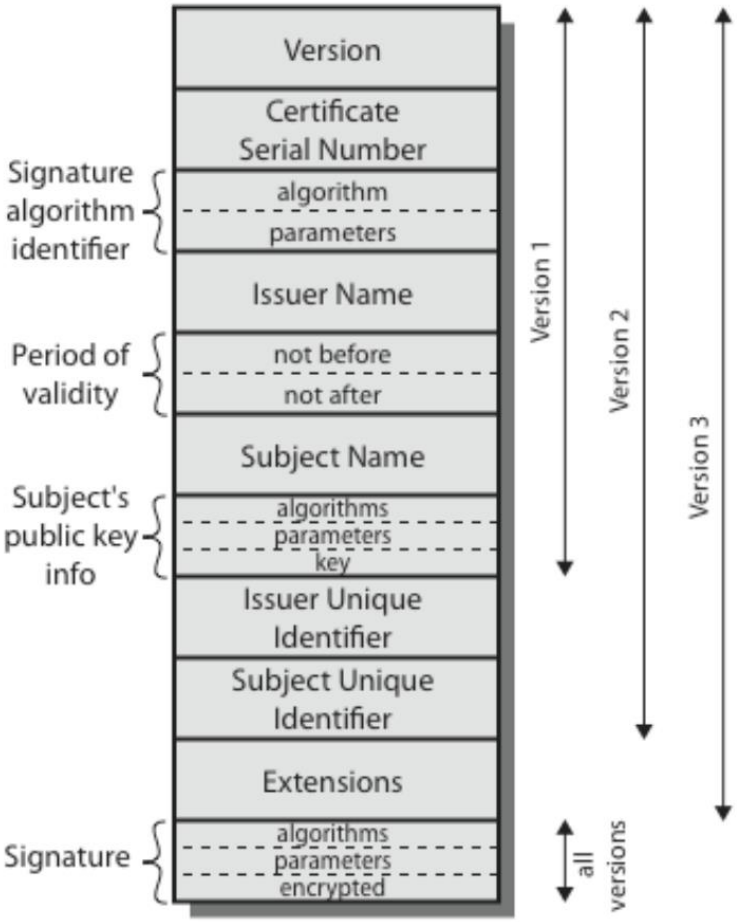
Digital (Public-Key) Certificate



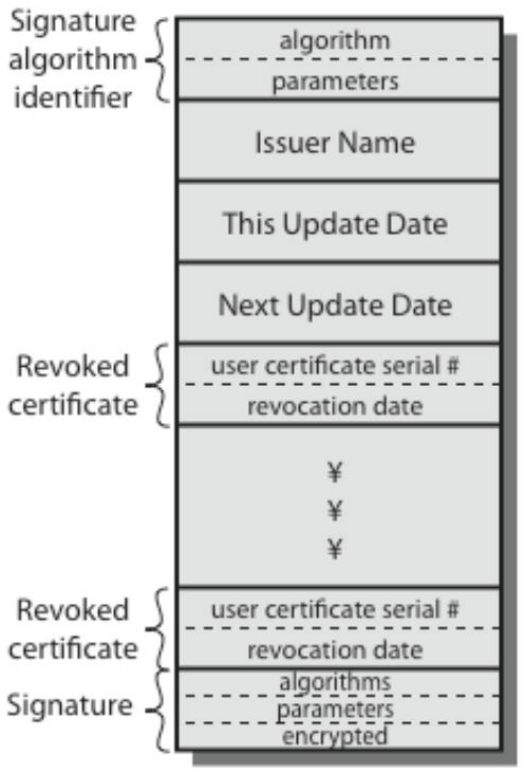
X.509 Digital Certificate

- **X.509** is a ***standard format*** of public key certificates defined by ITU (International Telecommunication Union)
- Widely used format for PKI (Public-Key Infrastructure)
- Define that public keys stored as certificates in a public directory
- Certificates are issued and **signed by CA**
- Used in several applications: SSL/TLS, IPSec, S/MIME, code signing

Structure of X.509 Digital Certificate

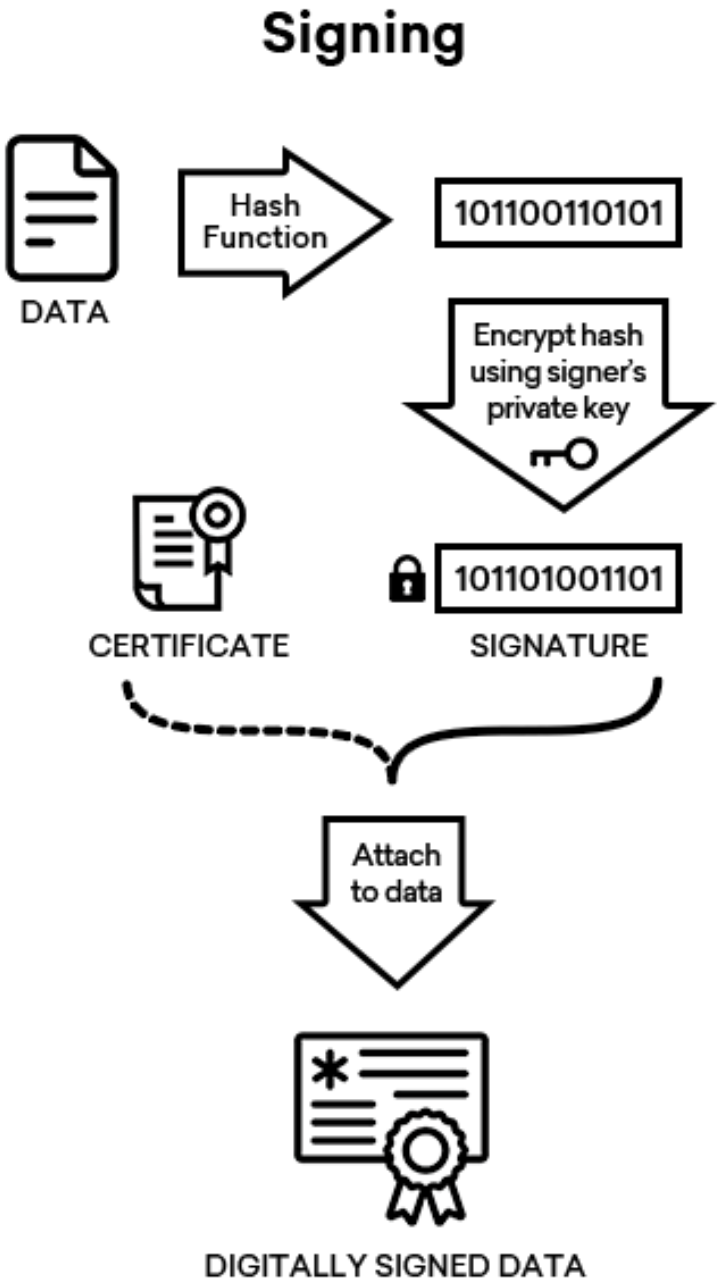


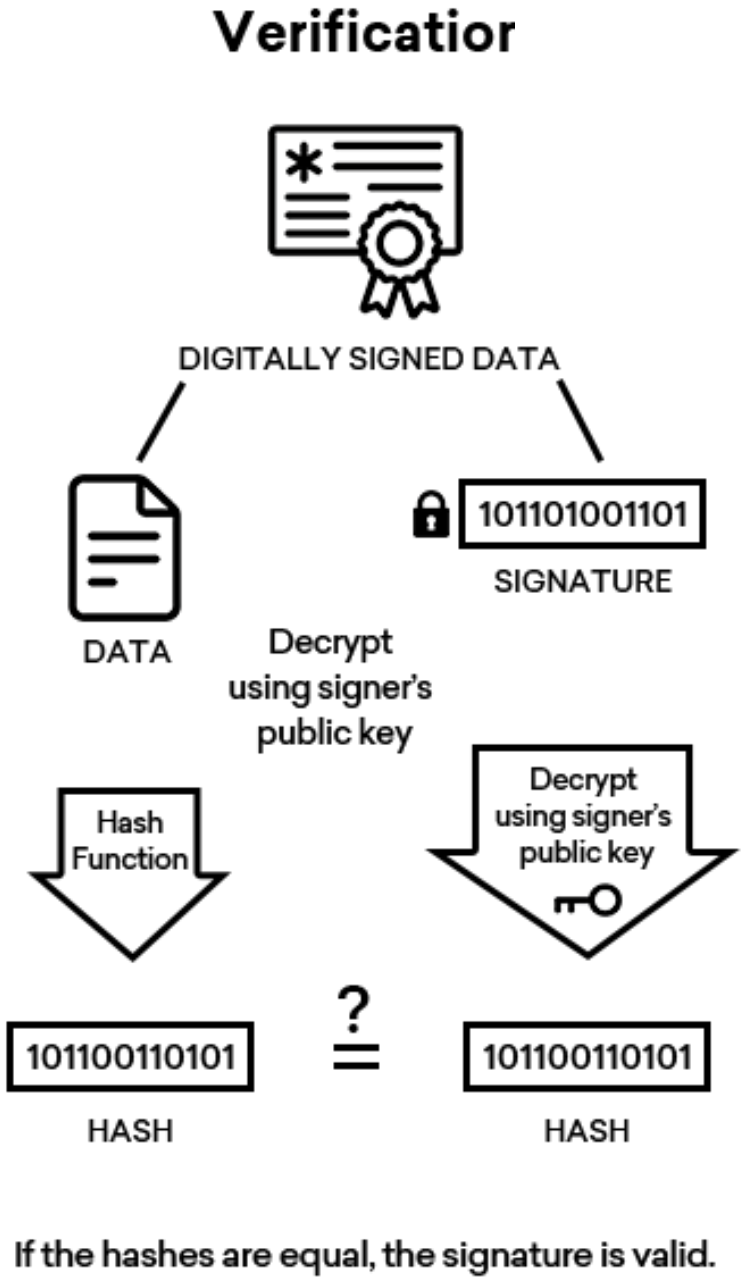
X.509 Certificate



Certificate Revocation List

How Digital Signature and Digital Certificate Works Together.





Summary

- **Hash function:** one-way function, transforming variable length input to fixed length output
  - Can be used for integrity check, message authentication, and digital signature
  - MD and SHA are examples of hash function
  - Attacks on hash functions: Birthday attack
- **Message authentication (MA):** ensure data integrity and authenticity using a <u>shared key</u>
  - Message Authentication Code (MAC)
  - Hash-based MAC (HMAC)
- **Digital signature:** ensure data integrity, authenticity, and *non-repudiation* using <u>PKE</u>
- **Digital certificate** (issued by CA): bind a public key to identity → public key trust anchor

มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วนโดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 10 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 10 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.